

GUJARAT TECHNOLOGICAL UNIVERSITY

LABORATORY MANUAL

Machine Vision

Subject Code: BE05041051

Program: Bachelor of Technology

Branch: Robotics and Automation

Semester: 5

w.e.f. Academic Year 2024-25

Student Name: _____

Enrollment No.: _____

Batch / Class: _____

INDEX

Sr. No.	Title of Practical	CO Mapped	Page No.	Date	Sign
1	Introduction to MATLAB/SCILAB: Basic Commands, Script Files and Image Reading/Writing	CO-1			
2	Study of Different Digital Image Classes, Types and File Formats	CO-1			
3	Arithmetic and Logical Operations on Images	CO-1			
4	Zooming, Shrinking and Bit-Plane Slicing of a Digital Image	CO-1			
5	Negative, Log, Power-Law and Contrast Stretching Transformations	CO-2			
6	Histogram Computation, Equalization and Matching/Specification	CO-2			
7	Image Smoothing and Sharpening Using Spatial Convolution Masks	CO-2			
8	1-D and 2-D Discrete Fourier Transform of a Digital Image	CO-2			
9	Frequency Domain Low-Pass and High-Pass Filtering (Ideal, Butterworth, Gaussian)	CO-2			
10	Image Degradation, Noise Modeling and Restoration Using Mean/Adaptive Filters	CO-3			
11	Morphological Operations: Dilation, Erosion, Opening and Closing	CO-5			

Practical 1: Introduction to MATLAB/SCILAB: Basic Commands, Script Files and Image Reading/Writing

Practical No.	1	Date	___ / ___ / ____
Competency & Practical Skills	Ability to set up the image processing environment and perform basic image I/O operations.		
Relevant CO	CO-1: Explain the fundamentals of digital image formation, human visual perception, image sensing, sampling, quantization and pixel relationships.		

Objectives

- To familiarize students with the MATLAB/SCILAB Image Processing environment.
- To learn basic commands, variables, and script file creation.
- To read, display and write digital images using built-in functions.

Theory

MATLAB (Matrix Laboratory) and SCILAB are numerical computing environments widely used for digital image processing because images can be represented as 2-D (grayscale) or 3-D (RGB) matrices of pixel intensity values.

A digital image $f(x,y)$ is a discretized, quantized representation of a continuous scene captured through an imaging sensor. Each element of the matrix, called a pixel, holds an intensity value (0-255 for an 8-bit image).

Basic functions used: `imread()` to load an image into a matrix, `imshow()/imagesc()` to display it, `imwrite()` to save it, `size()` to get dimensions, and `class()` to check the data type of the image.

Implementation (MATLAB / SCILAB Code)

```
% Experiment 1: Basic MATLAB commands and Image I/O
clc; clear all; close all;

% Read an image
I = imread('cameraman.tif');

% Display basic information
disp('Image size:');
disp(size(I));
disp('Image class:');
disp(class(I));

% Display the image
figure, imshow(I), title('Original Image');

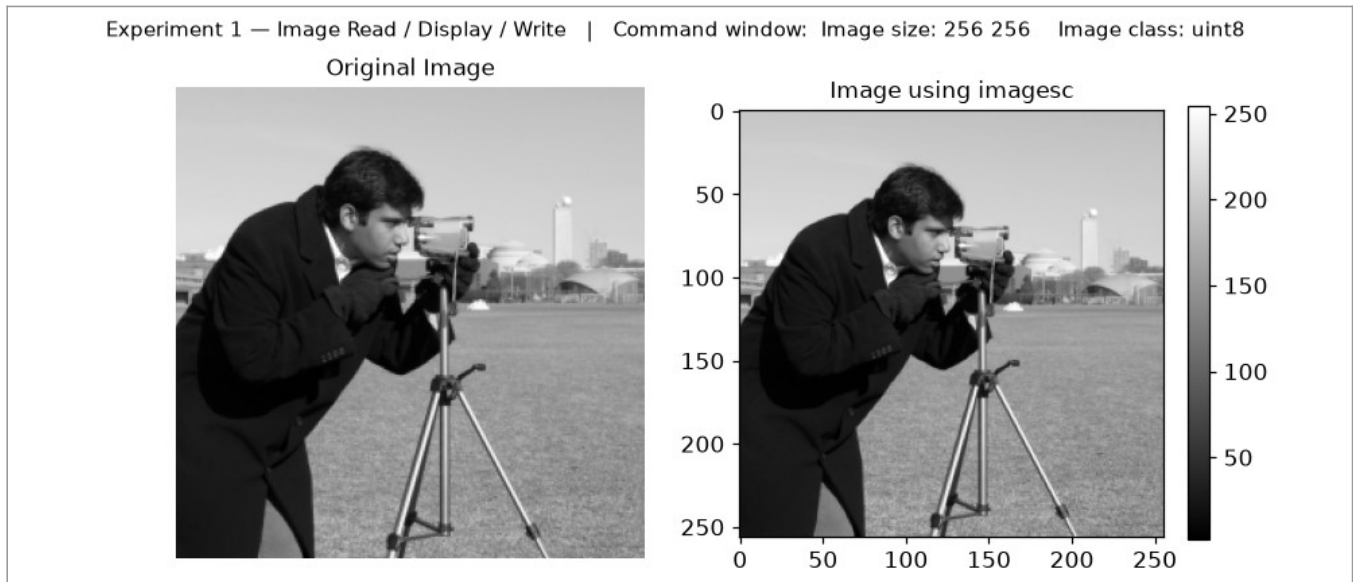
% Write/save the image in a different format
imwrite(I, 'output_image.png');

% Display image in grayscale using colormap
figure, imagesc(I), colormap(gray), title('Image using imagesc');
```

Result

The image was successfully read, its size and class were displayed in the command window, and the image was rendered in a figure window. A copy of the image was saved as output_image.png in the working directory.

Attach the output figure(s) / screenshot(s) generated during execution below:



Conclusion

The input image was read successfully and its size (256 x 256) and class (uint8) were displayed in the command window. The image was shown using both `imshow` and `imagesc` and then saved in a different format with `imwrite`. This experiment established the basic MATLAB image I/O workflow - reading an image into a matrix, displaying it and writing it back to disk - which is the foundation for all further image-processing work.

Quiz (Viva-Voce Questions)

1. What is the difference between `imshow()` and `imagesc()`?
2. What does the function `imread()` return for a color (RGB) image?
3. What is the default data type used to store an 8-bit grayscale image in MATLAB?
4. Differentiate between a script file and a function file.
5. What is the significance of the `clc`, `clear all`, and `close all` commands at the start of a script?

Reference Material List

- Gonzalez & Woods, Digital Image Processing, Addison-Wesley.
- Gonzalez & Woods, Digital Image Processing Using MATLAB, Addison-Wesley.
- MATLAB Image Processing Toolbox Documentation, MathWorks.

Evaluation Rubrics

Evaluation Criteria	Excellent (5)	Good (4)	Average (3)	Below Average (1-2)
Understanding of theory & concepts	☐	☐	☐	☐
Correctness of implementation / code	☐	☐	☐	☐
Accuracy & quality of results obtained	☐	☐	☐	☐
Analysis and conclusion drawn	☐	☐	☐	☐
Timely submission, viva & documentation	☐	☐	☐	☐
Total Marks Obtained (out of 25)				

Faculty Signature: _____ **Date:** ____ / ____ / _____

Practical 2: Study of Different Digital Image Classes, Types and File Formats

Practical No.	2	Date	___ / ___ / ____
Competency & Practical Skills	Ability to identify and convert between different image classes and file formats.		
Relevant CO	CO-1: Explain the fundamentals of digital image formation, human visual perception, image sensing, sampling, quantization and pixel relationships.		

Objectives

- To understand binary, indexed, grayscale and RGB image classes.
- To perform conversions between different image types.
- To study common image file formats and their properties.

Theory

Images can broadly be classified into four classes: Binary (1-bit, values 0 or 1), Indexed (uses a colormap/lookup table), Grayscale/Intensity (8-bit or 16-bit, values 0-255 or 0-65535), and RGB/True-color (three 8-bit planes for Red, Green, Blue).

Common file formats include BMP, TIFF, PNG, JPEG and GIF, which differ in compression scheme (lossy/lossless), bit depth support and metadata handling.

MATLAB provides conversion functions such as `rgb2gray()`, `im2bw()/imbinarize()`, `gray2ind()`, and `ind2rgb()` to move between these classes.

Implementation (MATLAB / SCILAB Code)

```
% Experiment 2: Image classes, types and file formats
clc; clear all; close all;

I_rgb = imread('peppers.png');
figure, imshow(I_rgb), title('RGB Image');

% Convert to grayscale
I_gray = rgb2gray(I_rgb);
figure, imshow(I_gray), title('Grayscale Image');

% Convert to binary
I_bin = imbinarize(I_gray);
figure, imshow(I_bin), title('Binary Image');

% Convert to indexed image
[I_ind, map] = rgb2ind(I_rgb, 64);
figure, imshow(I_ind, map), title('Indexed Image (64 colors)');

% Display class of each image
disp(class(I_rgb)); disp(class(I_gray));
disp(class(I_bin)); disp(class(I_ind));

% Save in different formats
imwrite(I_gray, 'gray.png');
imwrite(I_gray, 'gray.jpg');
imwrite(I_bin, 'binary.bmp');
```

Result

RGB, grayscale, binary and indexed versions of the same image were generated and displayed. Class types (uint8, logical) were verified in the command window, and the image was saved in PNG, JPEG and BMP formats for comparison.

Attach the output figure(s) / screenshot(s) generated during execution below:

Experiment 2 — Image Classes, Types and File Formats

RGB Image



Grayscale Image



Binary Image



Indexed Image (64 colors)



Conclusion

The same image was converted into RGB, grayscale, binary and indexed forms, and the class of each was verified in the command window. When saved, the JPEG file was clearly smaller than the PNG and BMP files, because JPEG uses lossy compression while PNG and BMP are lossless. We thus

understood the four image classes and the trade-off between file size and image quality for different file formats.

Quiz (Viva-Voce Questions)

6. What is the bit depth of a binary image?
7. Why does an indexed image require a colormap?
8. Which file format uses lossy compression by default?
9. How does `rgb2gray()` compute the grayscale intensity from R, G, B values?
10. What is the difference between `im2bw()` and `imbinarize()`?

Reference Material List

- Gonzalez & Woods, Digital Image Processing, Addison-Wesley.
- Scott E. Umbaugh, Computer Vision and Image Processing, Prentice-Hall.
- MATLAB Image Processing Toolbox Documentation, MathWorks.

Evaluation Rubrics

Evaluation Criteria	Excellent (5)	Good (4)	Average (3)	Below Average (1-2)
Understanding of theory & concepts	☐	☐	☐	☐
Correctness of implementation / code	☐	☐	☐	☐
Accuracy & quality of results obtained	☐	☐	☐	☐
Analysis and conclusion drawn	☐	☐	☐	☐
Timely submission, viva & documentation	☐	☐	☐	☐
Total Marks Obtained (out of 25)				

Faculty Signature: _____ **Date:** ____ / ____ / ____

Practical 3: Arithmetic and Logical Operations on Images

Practical No.	3	Date	___ / ___ / ____
Competency & Practical Skills	Ability to apply pixel-level arithmetic and logical operations for image manipulation.		
Relevant CO	CO-1: Explain the fundamentals of digital image formation, human visual perception, image sensing, sampling, quantization and pixel relationships.		

Objectives

- To perform arithmetic operations (addition, subtraction, multiplication) on images.
- To perform logical operations (AND, OR, XOR, NOT) on images.
- To observe the effect of these operations on image appearance.

Theory

Arithmetic operations are performed pixel-by-pixel between two images of the same size, or between an image and a scalar. Image addition is used for noise averaging, subtraction for change/motion detection, and multiplication/division for masking and shading correction.

Logical operations (AND, OR, XOR, NOT) operate on the binary representation of pixel values and are widely used for masking, region extraction and combining binary images.

Implementation (MATLAB / SCILAB Code)

```
% Experiment 3: Arithmetic and Logical Operations
clc; clear all; close all;

I1 = imread('cameraman.tif');
I2 = imread('rice.png');
I2 = imresize(I2, size(I1));

% Arithmetic operations
I_add = imadd(I1, I2);
I_sub = imsubtract(I1, I2);
I_mul = immultiply(I1, 1.5);
I_div = imdivide(I1, 2);

figure;
subplot(2,2,1), imshow(I_add), title('Addition');
subplot(2,2,2), imshow(I_sub), title('Subtraction');
subplot(2,2,3), imshow(I_mul), title('Multiplication');
subplot(2,2,4), imshow(I_div), title('Division');

% Logical operations (on binary images)
B1 = imbinarize(I1);
B2 = imbinarize(I2);

I_and = B1 & B2;
I_or = B1 | B2;
I_xor = xor(B1, B2);
I_not = ~B1;

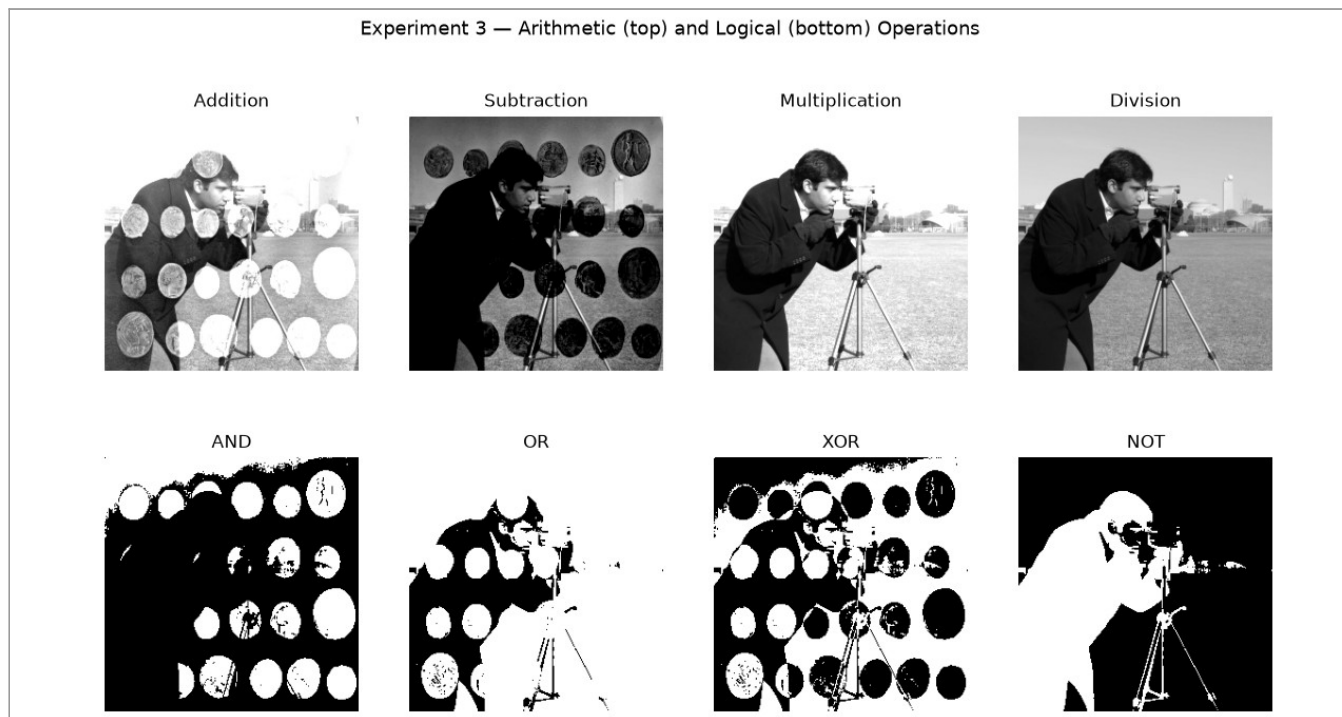
figure;
subplot(2,2,1), imshow(I_and), title('AND');
subplot(2,2,2), imshow(I_or), title('OR');
```

```
subplot(2,2,3), imshow(I_xor), title('XOR');
subplot(2,2,4), imshow(I_not), title('NOT');
```

Result

Arithmetic operations produced brightened/darkened and blended image outputs, while logical operations produced masked/combined binary images highlighting the overlapping and differing regions between the two source images.

Attach the output figure(s) / screenshot(s) generated during execution below:



Conclusion

Arithmetic operations changed the brightness and combined two images (addition brightens/blends, subtraction highlights differences), while logical operations worked on binary images. We observed that image subtraction is most suitable for change detection and the AND operation is most suitable for masking (extracting a region of interest). Pixel values were clipped to the 0-255 range whenever a result exceeded the limit.

Quiz (Viva-Voce Questions)

11. Why must two images be of the same size to perform arithmetic operations?
12. What is the practical use of image subtraction?
13. How does image averaging help in noise reduction?
14. What does the XOR operation highlight when applied to two binary images?
15. What happens to pixel values that exceed 255 during addition?

Reference Material List

- Gonzalez & Woods, Digital Image Processing, Addison-Wesley.
- A.K. Jain, Fundamentals of Digital Image Processing, PHI.
- MATLAB Image Processing Toolbox Documentation, MathWorks.

Evaluation Rubrics

Evaluation Criteria	Excellent (5)	Good (4)	Average (3)	Below Average (1-2)
Understanding of theory & concepts	☐	☐	☐	☐
Correctness of implementation / code	☐	☐	☐	☐
Accuracy & quality of results obtained	☐	☐	☐	☐
Analysis and conclusion drawn	☐	☐	☐	☐
Timely submission, viva & documentation	☐	☐	☐	☐
Total Marks Obtained (out of 25)				

Faculty Signature: _____ **Date:** ____ / ____ / ____

Practical 4: Zooming, Shrinking and Bit-Plane Slicing of a Digital Image

Practical No.	4	Date	___ / ___ / ____
Competency & Practical Skills	Ability to perform spatial resolution modification and bit-level decomposition of images.		
Relevant CO	CO-1: Explain the fundamentals of digital image formation, human visual perception, image sensing, sampling, quantization and pixel relationships.		

Objectives

- To zoom in and shrink a digital image using interpolation techniques.
- To decompose a grayscale image into its constituent bit planes.
- To analyze the contribution of each bit plane to overall image quality.

Theory

Zooming (up-sampling) increases image size using interpolation methods such as nearest-neighbour, bilinear or bicubic interpolation, whereas shrinking (down-sampling) reduces spatial resolution.

Bit-plane slicing decomposes an 8-bit grayscale image into 8 binary images, each representing one bit position. Higher-order bit planes (e.g., the MSB) contain the majority of visually significant information.

Implementation (MATLAB / SCILAB Code)

```
% Experiment 4: Zooming, Shrinking and Bit-Plane Slicing
clc; clear all; close all;

I = imread('cameraman.tif');

% Zooming (up-sampling)
I_zoom = imresize(I, 2, 'bicubic');
figure, imshow(I_zoom), title('Zoomed Image (2x)');

% Shrinking (down-sampling)
I_shrink = imresize(I, 0.5, 'nearest');
figure, imshow(I_shrink), title('Shrunk Image (0.5x)');

% Bit-plane slicing
I = double(I);
figure;
for bit = 0:7
    plane = mod(floor(I / (2^bit)), 2);
    subplot(2,4,bit+1);
    imshow(plane);
    title(['Bit Plane ', num2str(bit)]);
end
```

Result

Zoomed and shrunk versions of the image were generated using different interpolation techniques, and all eight bit planes were extracted and displayed, showing that the MSB planes carry most of the structural information.

Attach the output figure(s) / screenshot(s) generated during execution below:

Experiment 4 — Zooming, Shrinking and Bit-Plane Slicing

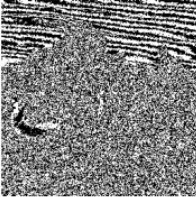
Zoomed Image (2x, bicubic)



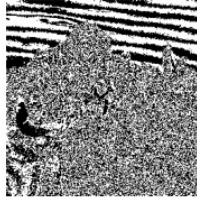
Shrunk Image (0.5x, nearest)



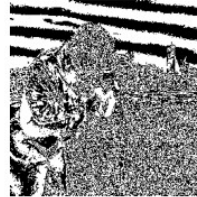
Bit Plane 0



Bit Plane 1



Bit Plane 2



Bit Plane 3



Bit Plane 4



Bit Plane 5



Bit Plane 6



Bit Plane 7



Conclusion

Zooming with bicubic interpolation produced a smoother enlarged image than nearest-neighbour, while shrinking caused a loss of fine detail. Bit-plane slicing showed that the higher-order planes (especially bits 6 and 7) carry most of the visually important information, whereas the lower planes mostly contain fine texture and noise. Hence the most-significant bit planes are the most important for image quality.

Quiz (Viva-Voce Questions)

16. What is the difference between nearest-neighbour and bicubic interpolation?
17. Why does shrinking an image cause information loss?
18. Which bit plane contributes the most to visual image quality and why?
19. How many bit planes exist in an 8-bit grayscale image?
20. What is the effect of removing the lower-order bit planes on image appearance?

Reference Material List

- Gonzalez & Woods, Digital Image Processing, Addison-Wesley.
- Milan Sonka, Machine Vision and Image Processing.
- MATLAB Image Processing Toolbox Documentation, MathWorks.

Evaluation Rubrics

Evaluation Criteria	Excellent (5)	Good (4)	Average (3)	Below Average (1-2)
Understanding of theory & concepts	☐	☐	☐	☐
Correctness of implementation / code	☐	☐	☐	☐
Accuracy & quality of results obtained	☐	☐	☐	☐
Analysis and conclusion drawn	☐	☐	☐	☐
Timely submission, viva & documentation	☐	☐	☐	☐
Total Marks Obtained (out of 25)				

Faculty Signature: _____ **Date:** ____ / ____ / _____

Practical 5: Negative, Log, Power-Law and Contrast Stretching Transformations

Practical No.	5	Date	___ / ___ / ____
Competency & Practical Skills	Ability to apply basic gray-level transformation functions for image enhancement.		
Relevant CO	CO-2: Apply spatial and frequency domain enhancement techniques to improve image quality.		

Objectives

- To implement image negative, log and power-law (gamma) transformations.
- To perform contrast stretching on a low-contrast image.
- To compare the visual effect of each transformation.

Theory

Gray-level transformations map each pixel intensity r to a new value $s = T(r)$. The image negative is given by $s = L - 1 - r$; log transformation by $s = c \cdot \log(1+r)$, which expands dark pixel values and compresses bright ones; and power-law (gamma) transformation by $s = c \cdot r^{\gamma}$, used for gamma correction in displays and cameras.

Contrast stretching is a piecewise-linear transformation that expands the range of intensity values in a low-contrast image to span the full dynamic range, thereby improving visual contrast.

Implementation (MATLAB / SCILAB Code)

```
% Experiment 5: Gray Level Transformations
clc; clear all; close all;

I = imread('pout.tif');
I = im2double(I);

% Negative transformation
I_neg = 1 - I;

% Log transformation
c = 1;
I_log = c * log(1 + I);

% Power-law (Gamma) transformation
gamma = 0.5;
I_gamma = I .^ gamma;

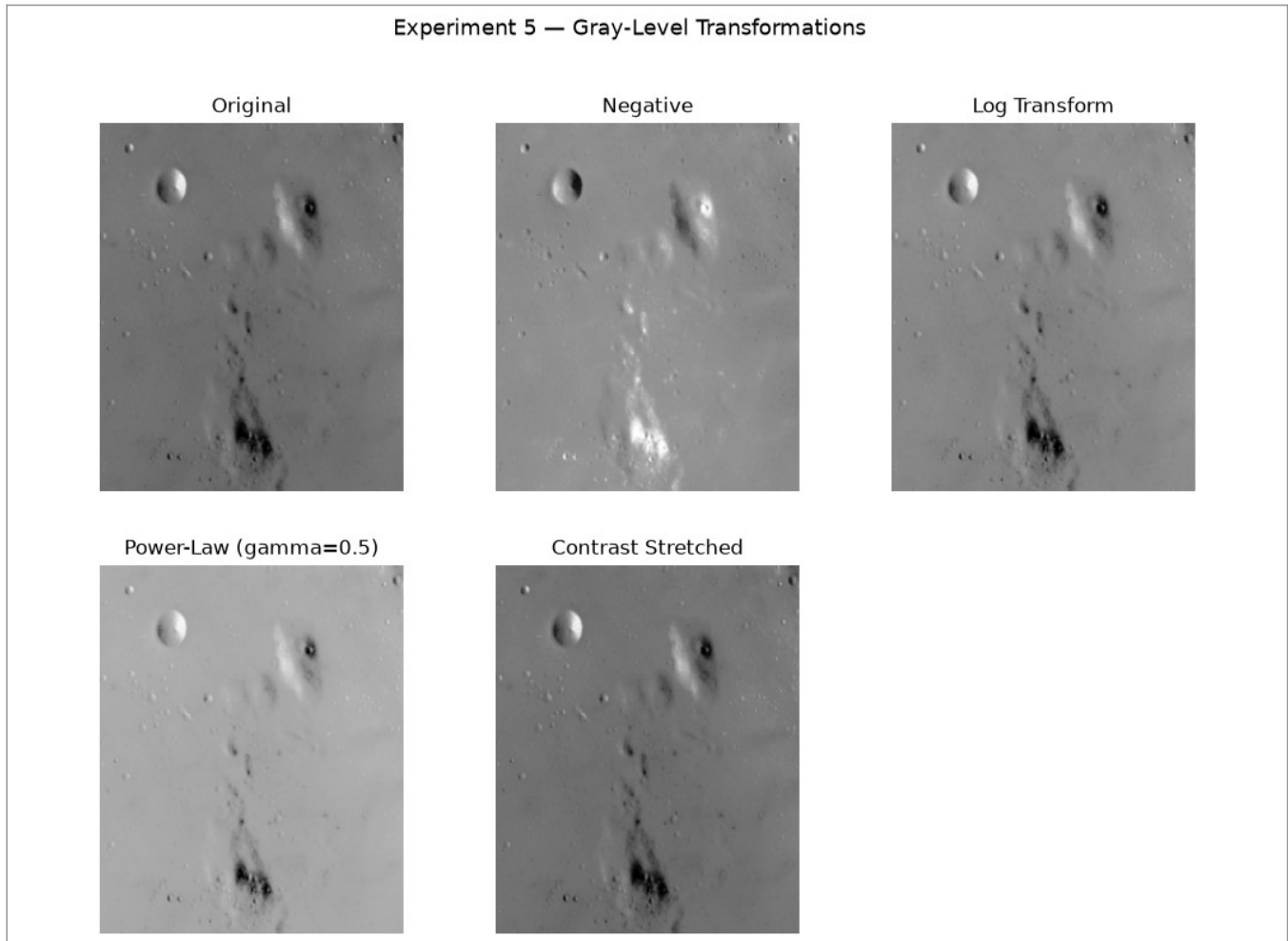
% Contrast stretching
r_min = min(I(:)); r_max = max(I(:));
I_stretch = (I - r_min) / (r_max - r_min);

figure;
subplot(2,3,1), imshow(I), title('Original');
subplot(2,3,2), imshow(I_neg), title('Negative');
subplot(2,3,3), imshow(I_log), title('Log Transform');
subplot(2,3,4), imshow(I_gamma), title('Power-Law (gamma=0.5)');
subplot(2,3,5), imshow(I_stretch), title('Contrast Stretched');
```

Result

All four transformations were applied to the input image. The negative transformation inverted the intensity, the log transform enhanced dark regions, the power-law transform adjusted overall brightness non-linearly, and contrast stretching visibly improved the dynamic range of the low-contrast input.

Attach the output figure(s) / screenshot(s) generated during execution below:



Conclusion

The negative transform inverted the intensities, the log transform brightened the dark regions, the power-law transform ($\gamma = 0.5$) brightened the image, and contrast stretching expanded the intensity range. A gamma less than 1 brightens an image while a gamma greater than 1 darkens it. For this low-contrast test image, contrast stretching and the gamma transform gave the best visual enhancement.

Quiz (Viva-Voce Questions)

21. Write the mathematical expression for the power-law transformation and explain the role of gamma.
22. Why is log transformation useful for images with a large dynamic range (e.g., Fourier spectra)?
23. What is the effect of $\gamma < 1$ versus $\gamma > 1$ on image brightness?
24. How does contrast stretching differ from histogram equalization?
25. What is meant by a piecewise-linear transformation function?

Reference Material List

- Gonzalez & Woods, Digital Image Processing, Addison-Wesley.
- Castleman K.R., Digital Image Processing, PHI.
- MATLAB Image Processing Toolbox Documentation, MathWorks.

Evaluation Rubrics

Evaluation Criteria	Excellent (5)	Good (4)	Average (3)	Below Average (1-2)
Understanding of theory & concepts	☐	☐	☐	☐
Correctness of implementation / code	☐	☐	☐	☐
Accuracy & quality of results obtained	☐	☐	☐	☐
Analysis and conclusion drawn	☐	☐	☐	☐
Timely submission, viva & documentation	☐	☐	☐	☐
Total Marks Obtained (out of 25)				

Faculty Signature: _____ **Date:** ____ / ____ / ____

Practical 6: Histogram Computation, Equalization and Matching/Specification

Practical No.	6	Date	___ / ___ / ____
Competency & Practical Skills	Ability to analyze and modify the intensity distribution of an image for enhancement.		
Relevant CO	CO-2: Apply spatial and frequency domain enhancement techniques to improve image quality.		

Objectives

- To compute and plot the histogram of a grayscale image.
- To perform histogram equalization to improve global contrast.
- To perform histogram matching against a specified reference histogram.

Theory

A histogram represents the frequency of occurrence of each intensity level in an image and provides a statistical description of its tonal distribution.

Histogram equalization redistributes intensity values so that the resulting histogram is approximately uniform, thereby improving overall image contrast. Histogram matching (specification) transforms the histogram of an image to match a desired/reference histogram, useful when a specific tonal appearance is required.

Implementation (MATLAB / SCILAB Code)

```
% Experiment 6: Histogram Processing
clc; clear all; close all;

I = imread('pout.tif');

% Plot histogram
figure, imhist(I), title('Original Histogram');

% Histogram Equalization
I_eq = histeq(I);
figure, imshow(I_eq), title('Histogram Equalized Image');
figure, imhist(I_eq), title('Equalized Histogram');

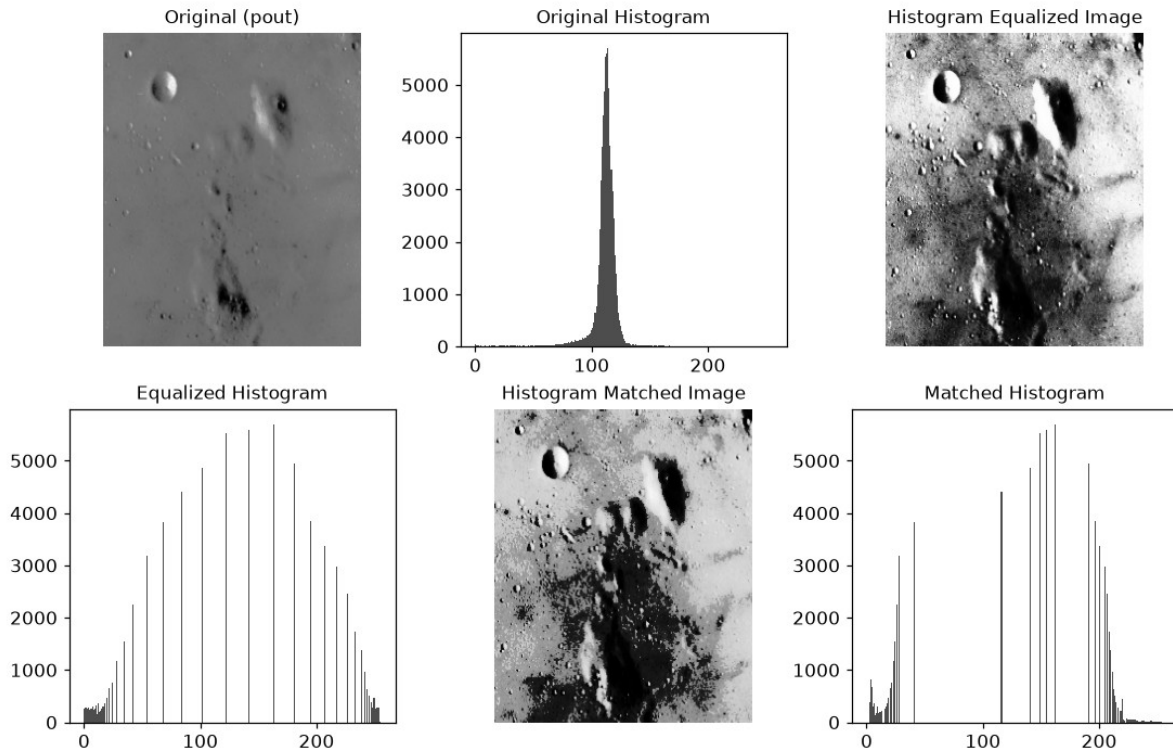
% Histogram Matching
ref = imread('cameraman.tif');
I_match = imhistmatch(I, ref);
figure, imshow(I_match), title('Histogram Matched Image');
figure, imhist(I_match), title('Matched Histogram');
```

Result

The original histogram showed a narrow intensity distribution (low contrast). After equalization, the histogram spread across the full range, producing a visibly higher-contrast image. Histogram matching reshaped the image histogram to resemble that of the reference image.

Attach the output figure(s) / screenshot(s) generated during execution below:

Experiment 6 — Histogram Equalization and Matching



Conclusion

The original image had a narrow histogram (low contrast). After equalization the histogram was spread across the full 0-255 range, giving a strong global contrast boost, while histogram matching reshaped the histogram to resemble the reference image. Equalization produced the larger visual improvement for this image, confirming that spreading the gray levels increases contrast.

Quiz (Viva-Voce Questions)

26. What does a histogram of an image represent?
27. Explain the mathematical basis of histogram equalization using the cumulative distribution function (CDF).
28. What is the difference between histogram equalization and histogram matching?
29. When would histogram matching be preferred over equalization?
30. Can histogram equalization over-enhance noise? Justify.

Reference Material List

- Gonzalez & Woods, Digital Image Processing, Addison-Wesley.
- A.K. Jain, Fundamentals of Digital Image Processing, PHI.
- MATLAB Image Processing Toolbox Documentation, MathWorks.

Evaluation Rubrics

Evaluation Criteria	Excellent (5)	Good (4)	Average (3)	Below Average (1-2)
Understanding of theory & concepts	☐	☐	☐	☐
Correctness of implementation / code	☐	☐	☐	☐
Accuracy & quality of results obtained	☐	☐	☐	☐
Analysis and conclusion drawn	☐	☐	☐	☐
Timely submission, viva & documentation	☐	☐	☐	☐
Total Marks Obtained (out of 25)				

Faculty Signature: _____ **Date:** ____ / ____ / ____

Practical 7: Image Smoothing and Sharpening Using Spatial Convolution Masks

Practical No.	7	Date	___ / ___ / ____
Competency & Practical Skills	Ability to design and apply spatial filter masks for image smoothing and sharpening.		
Relevant CO	CO-2: Apply spatial and frequency domain enhancement techniques to improve image quality.		

Objectives

- To implement smoothing (low-pass) spatial filters using convolution masks.
- To implement sharpening (high-pass) spatial filters using the Laplacian mask.
- To compare filtered outputs with the original image.

Theory

Spatial filtering involves moving a mask (kernel) over the image and computing a weighted sum (convolution/correlation) of pixel values under the mask. Smoothing filters (e.g., mean/averaging mask) reduce noise and blur edges, while sharpening filters (e.g., Laplacian mask) enhance edges and fine detail by emphasizing intensity discontinuities.

The Laplacian is a second-order derivative operator; adding the Laplacian response back to the original image produces edge-enhanced (sharpened) output.

Implementation (MATLAB / SCILAB Code)

```
% Experiment 7: Spatial Convolution - Smoothing and Sharpening
clc; clear all; close all;

I = imread('cameraman.tif');
I = im2double(I);

% Smoothing mask (3x3 averaging)
mean_mask = (1/9) * ones(3,3);
I_smooth = conv2(I, mean_mask, 'same');

% Sharpening using Laplacian mask
laplacian_mask = [0 1 0; 1 -4 1; 0 1 0];
I_lap = conv2(I, laplacian_mask, 'same');
I_sharp = I - I_lap;

figure;
subplot(1,3,1), imshow(I), title('Original');
subplot(1,3,2), imshow(I_smooth), title('Smoothed (Mean Filter)');
subplot(1,3,3), imshow(I_sharp), title('Sharpened (Laplacian)');
```

Result

The mean filter mask produced a visibly blurred/smoothed image with reduced noise, while the Laplacian-based sharpening mask enhanced edges and fine details, making structural boundaries more prominent.

Attach the output figure(s) / screenshot(s) generated during execution below:

Experiment 7 — Spatial Smoothing and Sharpening

Original



Smoothed (Mean Filter)



Sharpened (Laplacian)



Conclusion

The 3x3 mean filter blurred the image and reduced noise but also softened the edges, whereas the Laplacian mask sharpened the edges and fine detail. This demonstrates the basic trade-off in spatial filtering: smoothing reduces noise at the cost of edge sharpness, while sharpening strengthens edges but can also amplify noise.

Quiz (Viva-Voce Questions)

31. What is the difference between convolution and correlation in spatial filtering?
32. Why does a mean filter blur an image?
33. Explain how the Laplacian mask enhances edges.
34. What are order-statistics filters, and how do they differ from mean filters?
35. Why is the sum of coefficients in a smoothing mask usually equal to 1?

Reference Material List

- Gonzalez & Woods, Digital Image Processing, Addison-Wesley.
- Scott E. Umbaugh, Computer Vision and Image Processing, PHI.
- MATLAB Image Processing Toolbox Documentation, MathWorks.

Evaluation Rubrics

Evaluation Criteria	Excellent (5)	Good (4)	Average (3)	Below Average (1-2)
Understanding of theory & concepts	☐	☐	☐	☐

Evaluation Criteria	Excellent (5)	Good (4)	Average (3)	Below Average (1-2)
Correctness of implementation / code	☐	☐	☐	☐
Accuracy & quality of results obtained	☐	☐	☐	☐
Analysis and conclusion drawn	☐	☐	☐	☐
Timely submission, viva & documentation	☐	☐	☐	☐
Total Marks Obtained (out of 25)				

Faculty Signature: _____ **Date:** ____ / ____ / ____

Practical 8: 1-D and 2-D Discrete Fourier Transform of a Digital Image

Practical No.	8	Date	___ / ___ / ____
Competency & Practical Skills	Ability to transform and analyze images in the frequency domain.		
Relevant CO	CO-2: Apply spatial and frequency domain enhancement techniques to improve image quality.		

Objectives

- To compute the 1-D DFT of a signal/image row.
- To compute and visualize the 2-D DFT (frequency spectrum) of an image.
- To understand the significance of the frequency domain representation.

Theory

The Discrete Fourier Transform (DFT) converts a spatial-domain signal/image into its frequency-domain representation, revealing the frequency components that make up the image. For images, the 2-D DFT is computed using `fft2()`, and the zero-frequency (DC) component is shifted to the center of the spectrum using `fftshift()` for visualization.

Low frequencies correspond to smooth/slowly varying regions of the image, whereas high frequencies correspond to edges, noise and fine detail.

Implementation (MATLAB / SCILAB Code)

```
% Experiment 8: 1-D and 2-D DFT
clc; clear all; close all;

% 1-D DFT of a signal (row of an image)
I = imread('cameraman.tif');
row = double(I(128, :));
F1 = fft(row);
figure, plot(abs(F1)), title('1-D DFT Magnitude of a Row');

% 2-D DFT of the image
F2 = fft2(double(I));
F2_shifted = fftshift(F2);
magnitude = log(1 + abs(F2_shifted));

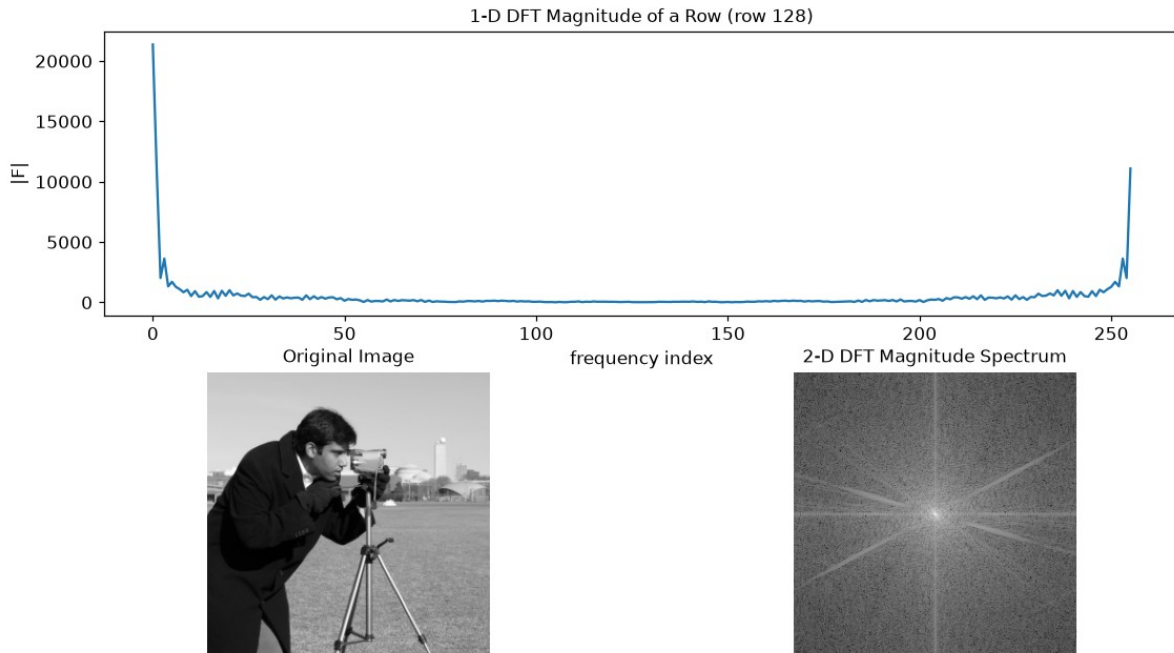
figure;
subplot(1,2,1), imshow(I), title('Original Image');
subplot(1,2,2), imshow(magnitude, []), title('2-D DFT Magnitude Spectrum');
```

Result

The 1-D DFT plot displayed the frequency content of a single image row, while the 2-D DFT magnitude spectrum (log-scaled and centered) revealed the concentration of low-frequency energy near the center and high-frequency components corresponding to edges spread outward.

Attach the output figure(s) / screenshot(s) generated during execution below:

Experiment 8 — 1-D and 2-D Discrete Fourier Transform



Conclusion

The 1-D DFT showed the frequency content of a single image row, and the 2-D DFT magnitude spectrum showed that most of the energy is concentrated at the centre (low frequencies / smooth areas), with the outer regions representing edges and fine detail. `fftshift()` was used to bring the zero-frequency (DC) component to the centre, and log-scaling was applied so that the weaker high-frequency values become visible.

Quiz (Viva-Voce Questions)

36. Why is `fftshift()` used after computing `fft2()`?
37. Why is a logarithmic transform applied to display the DFT magnitude spectrum?
38. What does the DC component (zero frequency) of an image represent?
39. State the periodicity property of the 2-D Fourier Transform.
40. Why is zero-padding sometimes required before computing the DFT?

Reference Material List

- Gonzalez & Woods, Digital Image Processing, Addison-Wesley.

- A.K. Jain, Fundamentals of Digital Image Processing, PHI.
- MATLAB Image Processing Toolbox Documentation, MathWorks.

Evaluation Rubrics

Evaluation Criteria	Excellent (5)	Good (4)	Average (3)	Below Average (1-2)
Understanding of theory & concepts	☐	☐	☐	☐
Correctness of implementation / code	☐	☐	☐	☐
Accuracy & quality of results obtained	☐	☐	☐	☐
Analysis and conclusion drawn	☐	☐	☐	☐
Timely submission, viva & documentation	☐	☐	☐	☐
Total Marks Obtained (out of 25)				

Faculty Signature: _____ **Date:** ____ / ____ / _____

Practical 9: Frequency Domain Low-Pass and High-Pass Filtering (Ideal, Butterworth, Gaussian)

Practical No.	9	Date	___ / ___ / ____
Competency & Practical Skills	Ability to design and apply frequency-domain filters for image smoothing and sharpening.		
Relevant CO	CO-2: Apply spatial and frequency domain enhancement techniques to improve image quality.		

Objectives

- To design Ideal, Butterworth and Gaussian low-pass filters in the frequency domain.
- To design the corresponding high-pass filters.
- To observe smoothing and sharpening effects of frequency domain filtering.

Theory

Frequency domain filtering involves multiplying the DFT of an image by a filter transfer function $H(u,v)$ and taking the inverse DFT of the result. Ideal filters have a sharp cutoff and cause ringing artifacts; Butterworth filters provide a smooth transition controlled by filter order; Gaussian filters provide the smoothest transition with no ringing.

Low-pass filters attenuate high frequencies (smoothing/blurring), while high-pass filters attenuate low frequencies (edge enhancement/sharpening).

Implementation (MATLAB / SCILAB Code)

```
% Experiment 9: Frequency Domain Filtering
clc; clear all; close all;

I = im2double(imread('cameraman.tif'));
[M, N] = size(I);
F = fftshift(fft2(I));

[u, v] = meshgrid(1:N, 1:M);
D0 = 50;
D = sqrt((u - N/2).^2 + (v - M/2).^2);

% Ideal Low Pass Filter
H_ideal = double(D <= D0);

% Butterworth Low Pass Filter (order n = 2)
n = 2;
H_butter = 1 ./ (1 + (D ./ D0).^(2*n));

% Gaussian Low Pass Filter
H_gauss = exp(-(D.^2) / (2 * (D0^2)));

% Apply filters
I_ideal = real(ifft2(ifftshift(F .* H_ideal)));
I_butter = real(ifft2(ifftshift(F .* H_butter)));
I_gauss = real(ifft2(ifftshift(F .* H_gauss)));

% High Pass = 1 - Low Pass
I_ideal_hp = real(ifft2(ifftshift(F .* (1 - H_ideal))));
```

```

I_gauss_hp = real(ifft2(ifftshift(F .* (1 - H_gauss))));

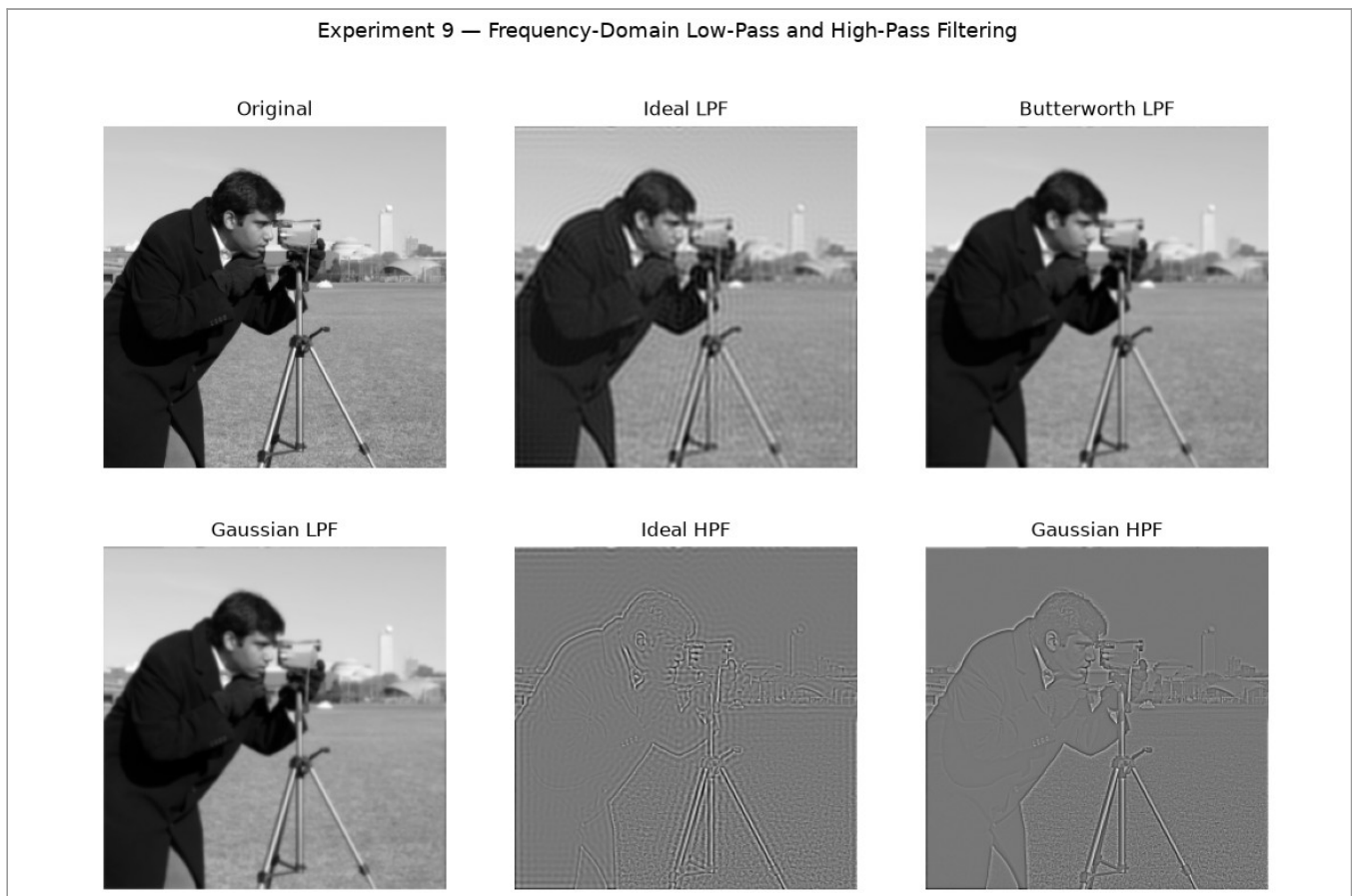
figure;
subplot(2,3,1), imshow(I), title('Original');
subplot(2,3,2), imshow(I_ideal, []), title('Ideal LPF');
subplot(2,3,3), imshow(I_butter, []), title('Butterworth LPF');
subplot(2,3,4), imshow(I_gauss, []), title('Gaussian LPF');
subplot(2,3,5), imshow(I_ideal_hp, []), title('Ideal HPF');
subplot(2,3,6), imshow(I_gauss_hp, []), title('Gaussian HPF');

```

Result

Low-pass filtered images appeared progressively smoother from Ideal to Gaussian, with the Ideal filter showing visible ringing artifacts near edges. High-pass filtered images highlighted edges and suppressed smooth/background regions.

Attach the output figure(s) / screenshot(s) generated during execution below:



Conclusion

The low-pass filters smoothed the image and the high-pass filters highlighted the edges. The Ideal filter produced visible ringing artifacts because of its sharp cut-off, the Butterworth filter gave a smoother result, and the Gaussian filter gave the smoothest result with no ringing. The Gaussian filter is therefore preferred in practice, as it avoids ringing while still smoothing the image.

Quiz (Viva-Voce Questions)

41. Why does the Ideal low-pass filter produce ringing artifacts?
42. How does the order (n) of a Butterworth filter affect the sharpness of the cutoff?
43. How is a high-pass filter derived from its corresponding low-pass filter?
44. What is unsharp masking and how does it relate to high-pass filtering?
45. Explain homomorphic filtering and its typical application.

Reference Material List

- Gonzalez & Woods, Digital Image Processing, Addison-Wesley.
- Milan Sonka, Machine Vision and Image Processing.
- MATLAB Image Processing Toolbox Documentation, MathWorks.

Evaluation Rubrics

Evaluation Criteria	Excellent (5)	Good (4)	Average (3)	Below Average (1-2)
Understanding of theory & concepts	☐	☐	☐	☐
Correctness of implementation / code	☐	☐	☐	☐
Accuracy & quality of results obtained	☐	☐	☐	☐
Analysis and conclusion drawn	☐	☐	☐	☐
Timely submission, viva & documentation	☐	☐	☐	☐
Total Marks Obtained (out of 25)				

Faculty Signature: _____ **Date:** ____ / ____ / ____

Practical 10: Image Degradation, Noise Modeling and Restoration Using Mean/Adaptive Filters

Practical No.	10	Date	___ / ___ / ____
Competency & Practical Skills	Ability to simulate image degradation and apply restoration techniques to recover image quality.		
Relevant CO	CO-3: Analyze image degradation models and apply appropriate restoration filters to recover degraded images.		

Objectives

- To add different types of noise (Gaussian, salt-and-pepper) to an image.
- To restore degraded images using arithmetic mean and adaptive filters.
- To compare restoration performance using PSNR/visual inspection.

Theory

Image degradation is commonly modeled as $g(x,y) = h(x,y) * f(x,y) + n(x,y)$, where f is the original image, h is the degradation function and n is additive noise. Noise models include Gaussian noise (arising from sensor/electronic noise) and salt-and-pepper (impulse) noise.

Restoration filters attempt to recover the original image: the arithmetic mean filter averages pixel neighborhoods (effective against Gaussian noise), the geometric mean filter smooths while preserving detail better, and adaptive filters adjust filtering strength based on local statistics (variance), performing better than fixed filters in regions with varying noise levels.

Implementation (MATLAB / SCILAB Code)

```
% Experiment 10: Noise Modeling and Restoration
clc; clear all; close all;

I = im2double(imread('cameraman.tif'));

% Add Gaussian noise
I_gauss_noise = imnoise(I, 'gaussian', 0, 0.02);

% Add Salt-and-Pepper noise
I_sp_noise = imnoise(I, 'salt & pepper', 0.05);

% Restoration using Arithmetic Mean Filter
h = fspecial('average', [3 3]);
I_restored_mean = imfilter(I_gauss_noise, h);

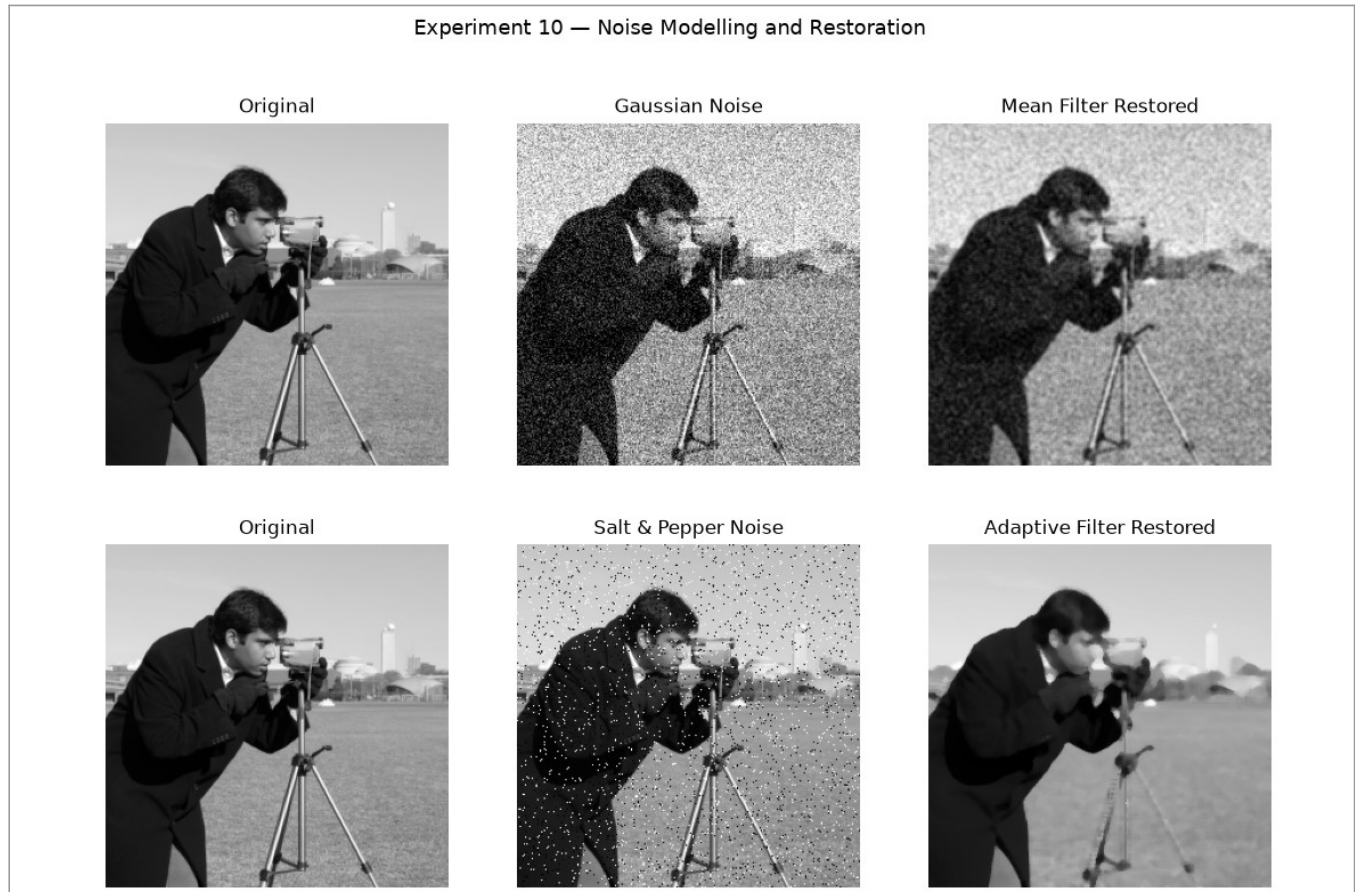
% Restoration using Adaptive (Wiener) Filter
I_restored_adaptive = wiener2(I_sp_noise, [5 5]);

figure;
subplot(2,3,1), imshow(I), title('Original');
subplot(2,3,2), imshow(I_gauss_noise), title('Gaussian Noise');
subplot(2,3,3), imshow(I_restored_mean), title('Mean Filter Restored');
subplot(2,3,4), imshow(I), title('Original');
subplot(2,3,5), imshow(I_sp_noise), title('Salt & Pepper Noise');
subplot(2,3,6), imshow(I_restored_adaptive), title('Adaptive Filter Restored');
```

Result

Gaussian and salt-and-pepper noise were successfully added to the test image. The arithmetic mean filter reduced Gaussian noise with some loss of sharpness, while the adaptive (Wiener) filter effectively suppressed impulse noise while better preserving edge details.

Attach the output figure(s) / screenshot(s) generated during execution below:



Conclusion

Gaussian and salt-and-pepper noise were added to the image and then restored. The arithmetic mean filter reduced the Gaussian noise but blurred the image, while the adaptive filter removed the salt-and-pepper noise much more effectively and preserved the edges. Adaptive filtering outperforms fixed averaging because it adjusts its action to the local image statistics instead of smoothing every region equally.

Quiz (Viva-Voce Questions)

46. What is the general image degradation/restoration model?
47. Differentiate between Gaussian noise and salt-and-pepper (impulse) noise.
48. Why is a median filter typically preferred over a mean filter for salt-and-pepper noise?
49. How does an adaptive filter adjust its behavior based on local image statistics?
50. What is periodic noise, and how can it be reduced using frequency-domain filtering?

Reference Material List

- Gonzalez & Woods, Digital Image Processing, Addison-Wesley.
- Castleman K.R., Digital Image Processing, PHI.
- MATLAB Image Processing Toolbox Documentation, MathWorks.

Evaluation Rubrics

Evaluation Criteria	Excellent (5)	Good (4)	Average (3)	Below Average (1-2)
Understanding of theory & concepts	☐	☐	☐	☐
Correctness of implementation / code	☐	☐	☐	☐
Accuracy & quality of results obtained	☐	☐	☐	☐
Analysis and conclusion drawn	☐	☐	☐	☐
Timely submission, viva & documentation	☐	☐	☐	☐
Total Marks Obtained (out of 25)				

Faculty Signature: _____ **Date:** ____ / ____ / ____

Practical 11: Morphological Operations: Dilation, Erosion, Opening and Closing

Practical No.	11	Date	___ / ___ / ____
Competency & Practical Skills	Ability to apply morphological operations for shape analysis on binary and grayscale images.		
Relevant CO	CO-5: Implement morphological image processing operations for binary and grayscale image analysis in machine vision applications.		

Objectives

- To implement dilation and erosion on binary and grayscale images.
- To implement opening and closing operations and study their applications.
- To observe the effect of structuring element shape and size on the result.

Theory

Morphological image processing analyzes and manipulates the geometric structure of objects within an image using a structuring element (SE). Dilation expands/grows the boundaries of foreground objects, while erosion shrinks them.

Opening (erosion followed by dilation) removes small objects/noise while preserving the shape and size of larger objects. Closing (dilation followed by erosion) fills small holes and gaps. The Hit-or-Miss transform is used for shape/pattern detection within binary images.

Implementation (MATLAB / SCILAB Code)

```
% Experiment 11: Morphological Operations
clc; clear all; close all;

I = imread('circles.png');
I = imbinarize(I);

se = strel('disk', 3);

% Dilation and Erosion
I_dilate = imdilate(I, se);
I_erode = imerode(I, se);

% Opening and Closing
I_open = imopen(I, se);
I_close = imclose(I, se);

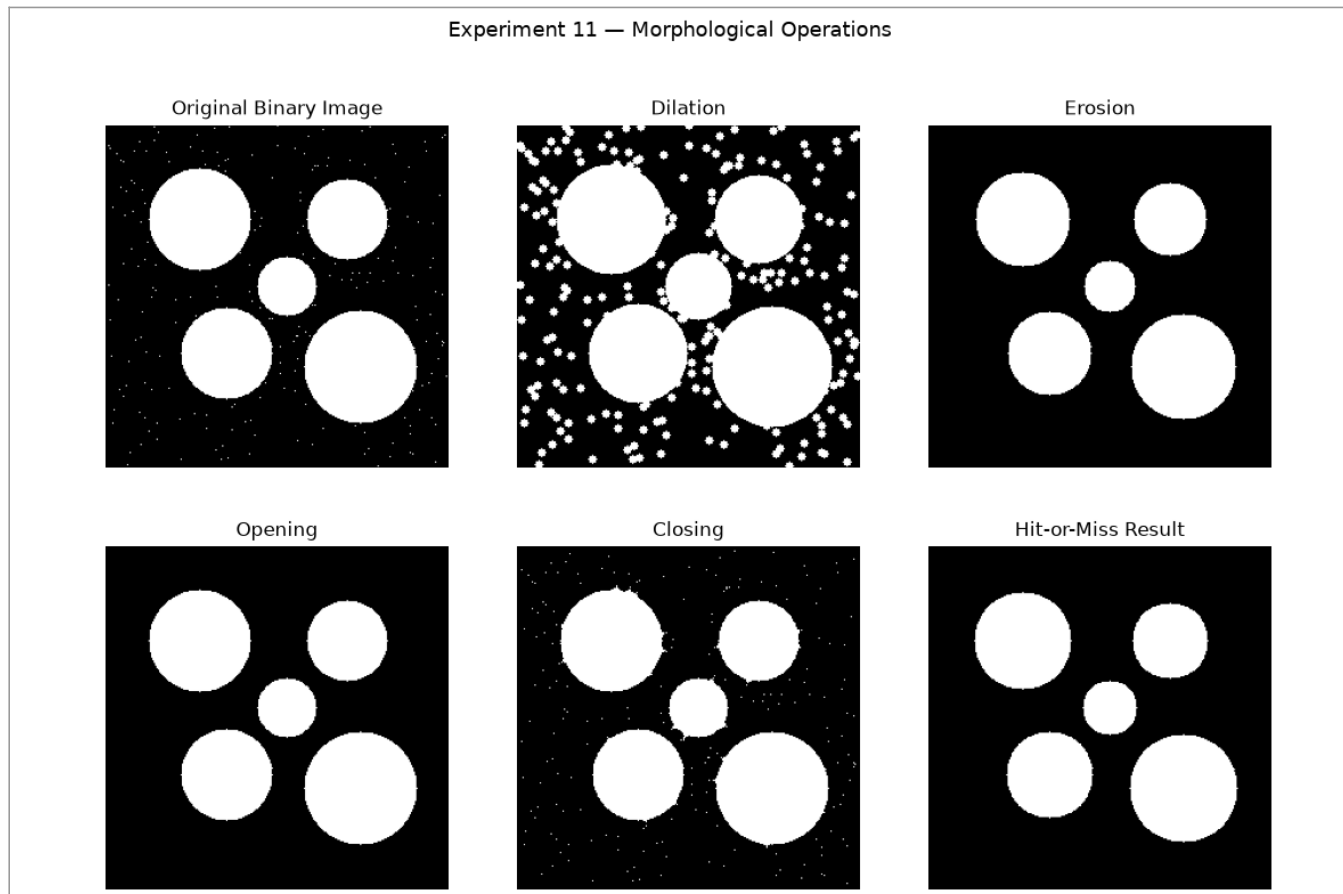
figure;
subplot(2,3,1), imshow(I), title('Original Binary Image');
subplot(2,3,2), imshow(I_dilate), title('Dilation');
subplot(2,3,3), imshow(I_erode), title('Erosion');
subplot(2,3,4), imshow(I_open), title('Opening');
subplot(2,3,5), imshow(I_close), title('Closing');

% Hit-or-Miss Transform
se_hit = strel('disk', 2);
I_hitmiss = bwhitmiss(I, se_hit);
subplot(2,3,6), imshow(I_hitmiss), title('Hit-or-Miss Result');
```

Result

Dilation enlarged the foreground regions while erosion shrank them. Opening removed small isolated noise particles, and closing filled small gaps/holes within the objects, confirming standard morphological behavior on the test image.

Attach the output figure(s) / screenshot(s) generated during execution below:



Conclusion

Dilation enlarged the foreground objects and filled small gaps, erosion shrank them and removed small specks, opening removed small noise while keeping the object size, and closing filled small holes inside the objects. A larger structuring element produced a greater change. Typical applications: dilation - joining broken parts, erosion - removing small noise, opening - cleaning noise, closing - filling holes.

Quiz (Viva-Voce Questions)

51. Define dilation and erosion mathematically using set theory.
52. Why is opening used for noise removal rather than erosion alone?
53. How does the choice of structuring element (disk, square, line) affect the output?
54. What is the Hit-or-Miss transform used for?
55. How are morphological operations extended from binary to grayscale images?

Reference Material List

- Gonzalez & Woods, Digital Image Processing, Addison-Wesley.
- Scott E. Umbaugh, Computer Vision and Image Processing, PHI.
- MATLAB Image Processing Toolbox Documentation, MathWorks.

Evaluation Rubrics

Evaluation Criteria	Excellent (5)	Good (4)	Average (3)	Below Average (1-2)
Understanding of theory & concepts	☐	☐	☐	☐
Correctness of implementation / code	☐	☐	☐	☐
Accuracy & quality of results obtained	☐	☐	☐	☐
Analysis and conclusion drawn	☐	☐	☐	☐
Timely submission, viva & documentation	☐	☐	☐	☐
Total Marks Obtained (out of 25)				

Faculty Signature: _____ **Date:** ____ / ____ / ____